#19 - GraphRAG: Từ văn bản tới Knowledge Graph

Bài viết này dành cho ai?

Bài viết này dành cho người đã nắm các thao tác Python cơ bản, có kiến thức nền tảng về RAG và muốn tìm hiểu một cách tiếp cận nâng cao hơn trong xây dựng hệ thống RAG.

Nội dung bài viết sẽ tập trung vào GraphRAG và thực hành xây dựng đồ thị tri thức từ văn bản, tổ chức thông tin trong đồ thị và khai thác cấu trúc này trong quá trình truy xuất.

Sau bài viết, bạn sẽ hiểu:

- GraphRAG là gì và vì sao cấu trúc đồ thị có thể bổ sung cho cách truy xuất dựa trên vector.
- Cách thông tin trong văn bản được tổ chức thành entity, relationship, community và community report.
- Cách knowledge graph được xây dựng từ dữ liệu văn bản và được sử dụng trong quá trình truy xuất.
- Local Search và Global Search trong Microsoft GraphRAG.

Tác giả: Kỹ sư AI Hoàng Duy Hưng
AI Guru x TiniX

MỤC LỤC

Bảng thuật ngữ **3**
1 GraphRAG là gì? **4**
2 Microsoft GraphRAG **4**
3 Thực hành với Microsoft GraphRAG **5**
4 Chuẩn bị môi trường **6**
5 Cài đặt chi tiết **6**
5.1 Cài đặt Microsoft GraphRAG 6
5.2 Chuẩn bị workspace GraphRAG 7
5.3 Chuẩn bị dữ liệu 7
5.4 Cấu hình GraphRAG 10
5.5 Lập chỉ mục cho dữ liệu 12
5.5.1 Xác định thực thể và mối quan hệ 13
5.5.2 Tạo Community từ đồ thị tri thức 15
5.5.3 Community report được tạo như thế nào? 16
5.5.4 Thực hiện chạy bước lập chỉ mục 16
5.5.5 Kết quả lập chỉ mục 17
5.6 Truy vấn với Local Search và Global Search 18
5.6.1 Local Search 18
5.6.2 Global Search 19
6 Ví dụ khi GraphRAG có lợi thế **20**
6.1 Case 1: Chữ ký điện tử và hợp đồng gửi qua mạng 20
6.2 Case 2: Thử việc, BHXH và công đoàn 21
6.3 Case 3: Hợp đồng mang tên “cộng tác viên” 22
7 Đánh giá khả năng truy xuất **23**
7.1 Phương pháp truy xuất 24
7.2 Thực hiện đánh giá 25
7.3 Chỉ số đánh giá 25
7.4 Kết quả 26
8 Tổng kết **26**
Nguồn tham khảo **26**

Bảng thuật ngữ

Thuật ngữ	Cách hiểu tiếng Việt	Giải thích ngắn
RAG	Sinh câu trả lời có truy xuất	Quy trình tìm các đoạn tài liệu liên quan rồi đưa chúng cho model để tạo câu trả lời có căn cứ.
Corpus	Tập tài liệu nguồn	Toàn bộ dữ liệu được đưa vào hệ thống. Corpus trong bài là 63 văn bản pháp luật lao động lấy từ bộ dữ liệu YuITC.
LLM	Mô hình ngôn ngữ lớn	Model đọc chunk, trích graph, viết report và sinh câu trả lời. Output của LLM có thể sai dù câu văn trôi chảy.
Chunk	Đoạn văn nhỏ	Phần nội dung được tách ra từ document để embedding, extraction và truy xuất.
Embedding	Vector biểu diễn nội dung	Dãy số đại diện cho ý nghĩa của một đoạn văn, dùng trong vector search.
Entity	Thực thể	Một đối tượng được nhắc đến trong văn bản, ví dụ Tòa án, thẻ bảo hiểm y tế hoặc hợp đồng lao động.
Knowledge graph	Đồ thị tri thức	Mạng gồm các entity làm node và relationship làm cạnh.
Community	Cụm trong graph	Nhóm entity và relationship có liên hệ gần nhau, được thuật toán phân cụm gom lại.
Community report	Báo cáo của một cụm	Bản tóm tắt do LLM tạo từ entity và relationship trong một community.
Indexing	Lập chỉ mục	Toàn bộ quá trình đọc tài liệu, chia chunk, trích graph, tạo community, report và embedding.
Local search	Tìm quanh thực thể	Bắt đầu từ entity liên quan đến câu hỏi rồi mở rộng sang relationship, chunk và community gần nó.
Global search	Tổng hợp toàn corpus	Dùng nhiều community report và cơ chế map-reduce để trả lời câu hỏi cần cái nhìn tổng thể.
Map-reduce	Chia nhỏ rồi tổng hợp	Bước map xử lý từng nhóm dữ liệu; bước reduce hợp nhất các kết quả trung gian thành câu trả lời cuối.

Thuật ngữ	Cách hiểu tiếng Việt	Giải thích ngắn
Parquet	File dữ liệu dạng cột	Định dạng lưu bảng hiệu quả; GraphRAG dùng để lưu documents, entities, relationships và reports.

1. GraphRAG là gì?

Trong kỹ thuật RAG, câu hỏi được mã hóa thành vector để tìm các đoạn văn bản liên quan trong kho tài liệu. Cách tiếp cận này phù hợp khi câu trả lời nằm trong một hoặc một vài đoạn văn bản liên quan trực tiếp đến câu hỏi.

Nhưng khó khăn sẽ xuất hiện khi câu hỏi cần sử dụng thông tin nằm ở nhiều phần khác nhau của tài liệu. Ví dụ, để trả lời câu hỏi:

> *"Người thuê nhà có được tự ý chấm dứt hợp đồng khi căn nhà bị hư hỏng nghiêm trọng hay không?"*

hệ thống có thể cần đồng thời tìm được quy định về nghĩa vụ sửa chữa của bên cho thuê, điều kiện đơn phương chấm dứt hợp đồng và quyền yêu cầu bồi thường của bên thuê. Các thông tin này có thể nằm ở những đoạn văn bản khác nhau và không nhất thiết đều có độ tương đồng ngữ nghĩa cao với câu hỏi.

GraphRAG được phát triển để hỗ trợ những bài toán như vậy bằng cách bổ sung cấu trúc đồ thị vào quá trình tổ chức và truy xuất thông tin. Thay vì chỉ dựa vào các đoạn văn bản và vector, GraphRAG còn biểu diễn các đối tượng và mối quan hệ giữa chúng, từ đó hỗ trợ việc kết nối và tổng hợp thông tin phân tán trong tập tài liệu.

2. Microsoft GraphRAG

Microsoft GraphRAG là một triển khai của GraphRAG do Microsoft phát triển. Bên cạnh việc sử dụng các vector biểu diễn, hệ thống còn tổ chức thông tin trong tài liệu dưới dạng một đồ thị tri thức (knowledge graph) để hỗ trợ quá trình truy xuất.

Trong quá trình lập chỉ mục, Microsoft GraphRAG tạo ra một số loại dữ liệu chính:

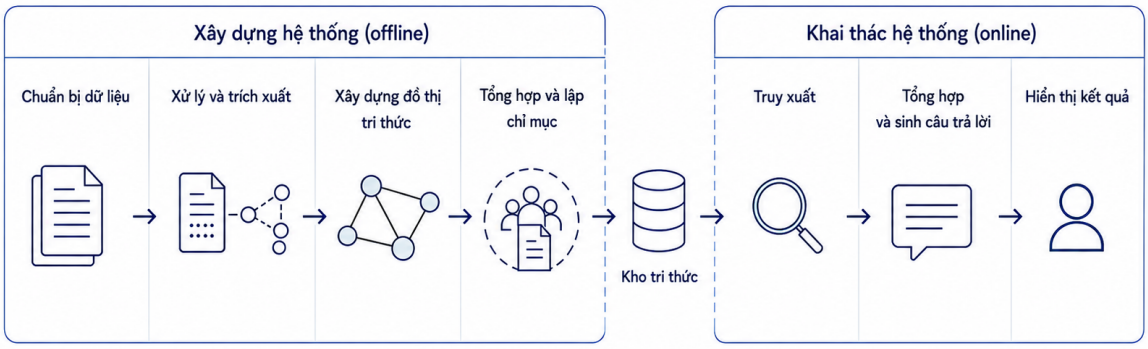
- **Entity**: các đối tượng hoặc khái niệm quan trọng được phát hiện trong văn bản;
- **Relationship**: mối liên hệ giữa các entity;
- **Community**: các nhóm entity có liên kết chặt chẽ với nhau trong knowledge graph;
- **Community report**: nội dung tổng hợp thông tin của từng community;

- **Embedding**: vector biểu diễn một số nội dung trong hệ thống để phục vụ truy xuất theo độ tương đồng.

Những dữ liệu này được sử dụng theo các cách khác nhau khi truy vấn. Chẳng hạn, Local Search khai thác thông tin quanh các entity liên quan đến câu hỏi, còn Global Search sử dụng các community report để tổng hợp thông tin trên phạm vi rộng hơn của corpus.

3. Thực hành với Microsoft GraphRAG

Trong bài viết này, mình sẽ xây dựng một hệ thống Microsoft GraphRAG trên tập văn bản pháp luật lao động. Từ dữ liệu nguồn, hệ thống sẽ xây dựng đồ thị tri thức và sử dụng đồ thị này để hỗ trợ quá trình truy xuất.



Hình 1: Tổng quan pipeline triển khai GraphRAG.

Nhìn chung, chúng ta sẽ thực hiện:

1. Chuẩn bị dữ liệu đầu vào;
2. Cấu hình và chạy quá trình xây dựng chỉ mục;
3. Kiểm tra đồ thị và dữ liệu được tạo ra;
4. Chạy thử phương thức truy xuất của Microsoft GraphRAG;
5. Đánh giá và so sánh kết quả truy xuất.

4. Chuẩn bị môi trường

Phần thực hành sử dụng Python 3.12 và Microsoft GraphRAG 3.1.1. Mô hình ngôn ngữ được gọi thông qua OpenRouter. Mô hình embedding BGE-M3 được sử dụng qua một API.

Thành phần	Cấu hình sử dụng
Python	3.12.4
Microsoft GraphRAG	3.1.1
Mô hình ngôn ngữ	`Deepseek-V4-Flash`
Mô hình embedding	`BGE-M3`

Bảng 2: Môi trường sử dụng trong phần thực hành

Khóa API của OpenRouter được lưu trong biến `OPENROUTER_API_KEY` của file `.env`. Các tham số còn lại được khai báo trong `settings.yaml`.

Cấu trúc ban đầu của dự án như sau:

```
graphrag-vietnamese-legal/
 |-- .env.example
 |-- .gitignore
 |-- requirements.txt
 |-- settings.yaml.example
 |-- corpus/
 |-- prompts/
 |-- scripts/
 |-- queries/
 |-- graph/
 `-- embeddings/

```

Trong đó, `corpus/` lưu dữ liệu đầu vào, `prompts/` lưu prompt, `scripts/` chứa runner và chương trình đánh giá. Hai thư mục `graph/` và `embeddings/` chứa artefact đã được lưu kèm project để có thể kiểm tra và tái lập benchmark mà không phải gọi lại LLM ngay từ đầu.

5. Cài đặt chi tiết

5.1 Cài đặt Microsoft GraphRAG

Sau khi kích hoạt môi trường Python của project, cài đặt các thư viện cần thiết:

```
python -m pip install -r requirements.txt
```

Dự án này sử dụng Microsoft GraphRAG phiên bản 3.1.1. Bạn có thể kiểm tra phiên bản sau khi cài đặt bằng lệnh:

```
graphrag --version
```

Kết quả cần hiển thị phiên bản `3.1.1`.

5.2 Chuẩn bị workspace GraphRAG

Microsoft GraphRAG gọi thư mục chứa `settings.yaml`, `.env`, input, prompt và output là workspace.

Tạo cấu hình chạy từ hai file mẫu bằng lệnh:

```
cp .env.example .env
 cp settings.yaml.example settings.yaml
```

Sau đó thay giá trị mẫu của `OPENROUTER_API_KEY` trong `.env` bằng khóa thật. Giữ `EMBEDDING_API_KEY=unused` nếu endpoint embedding không yêu cầu xác thực. Hai file vừa tạo có vai trò:

- `.env`: lưu các biến môi trường và API key;
- `settings.yaml`: chứa cấu hình của GraphRAG;
- `prompts/`: đã có sẵn trong project và được tham chiếu từ cấu hình.

5.3 Chuẩn bị dữ liệu

Trong dự án, ta sẽ sử dụng bộ dữ liệu `YuITC/Vietnamese-Legal-Documents` được công bố trên Hugging Face. Đây là bộ dữ liệu dành cho bài toán truy xuất thông tin pháp luật tiếng Việt, mỗi câu hỏi được đi kèm với một hoặc nhiều văn bản pháp luật được gắn nhãn là có liên quan.

Dataset gồm một tập văn bản pháp luật và các tập câu hỏi dùng cho huấn luyện và đánh giá. Trong bài này, chúng ta sử dụng tập `test` để vừa tạo một tập dữ liệu nhỏ cho GraphRAG, vừa giữ lại các câu hỏi và nhãn liên quan để đánh giá khả năng truy xuất ở các bước sau.

Mỗi mẫu trong tập `test` có các trường chính:

- `qid`: mã định danh của câu hỏi;
- `question`: câu hỏi pháp luật bằng tiếng Việt;
- `cid`: danh sách mã của các văn bản được gắn nhãn liên quan;

- `context_list`: nội dung của các văn bản tương ứng với từng `cid`.

Có thể hình dung một mẫu dữ liệu theo dạng:

```
{
  "qid": 78013,
  "question": "Có được lồng ghép thỏa thuận thử việc
              trong hợp đồng lao động không?",
  "cid": [62671, 61862],
  "context_list": [
    "... nội dung văn bản 62671 ...",
    "... nội dung văn bản 61862 ..."
  ]
}
```

Trong ví dụ trên, câu hỏi có hai văn bản được gắn nhãn liên quan. Phần tử đầu tiên của `context_list` là nội dung của `cid` đầu tiên, phần tử thứ hai tương ứng với `cid` thứ hai. Nhờ đó, từ các mẫu câu hỏi có thể khôi phục lại từng văn bản pháp luật độc lập.

Chọn dữ liệu cho bài thực hành

Toàn bộ dataset này khá lớn, vì vậy chúng ta chỉ lấy một tập con liên quan đến pháp luật lao động. Các câu hỏi được chọn khi:

- Có từ hai đến năm văn bản liên quan;
- Nội dung chứa "thử việc" hoặc "hợp đồng lao động".

Có 38 câu hỏi thỏa các điều kiện trên. Các `cid` xuất hiện trong 38 câu hỏi được hợp nhất và loại bỏ trùng lặp, thu được 63 văn bản pháp luật duy nhất. Đây là dữ liệu sẽ được sử dụng để xây dựng GraphRAG.

Dữ liệu lúc này được tách thành hai phần:

- 63 văn bản pháp luật được dùng để xây dựng chỉ mục và đồ thị tri thức;
- 38 câu hỏi cùng danh sách `cid` liên quan được giữ riêng để đánh giá pha truy xuất thông tin.

Chuyển văn bản sang định dạng đầu vào

Trong dataset gốc, mỗi bản ghi tương ứng với một câu hỏi và chứa danh sách các văn bản pháp luật liên quan trong `context_list`. Cấu trúc này phù hợp cho bài toán đánh giá retrieval nhưng chưa phù hợp để đưa trực tiếp vào GraphRAG, vì GraphRAG cần xử lý từng văn bản như một tài liệu độc lập.

Do đó, các văn bản trong `context_list` được tách ra, gắn lại với `cid` tương ứng và loại bỏ các văn bản trùng lặp. Sau bước này thu được 63 văn bản duy nhất, được lưu thành một file CSV trong đó mỗi dòng tương ứng với một tài liệu:

```
id,title,text,source
CID-62671,"CID 62671: Điều 24. Thử việc",
"CID: 62671 ...",
"YuITC/Vietnamese-Legal-Documents"
```

Mỗi tài liệu gồm bốn trường:

- `id`: mã định danh được tạo từ `cid`;
- `title`: tiêu đề giúp nhận diện văn bản;
- `text`: nội dung pháp luật được đưa vào GraphRAG;
- `source`: thông tin về nguồn dữ liệu.

Trong đó, cột `text` là nội dung được GraphRAG chia thành các chunk và sử dụng trong quá trình xây dựng đồ thị. `id` được giữ lại để kết quả truy xuất sau này có thể đối chiếu với `cid` trong tập đánh giá.

Cấu hình đọc file CSV trong `settings.yaml` như sau:

```
input:
  type: csv
  storage:
    type: file
    base_dir: corpus
  id_column: id
  title_column: title
  text_column: text
  file_pattern: "legal_labor\\.csv$"
```

Trong 38 câu hỏi dùng để chọn corpus, project lưu 26 câu hỏi tình huống dùng cho benchmark minh họa tại `scripts/generated_candidate_queries.jsonl`:

```
{"qid":"78013",
 "question":"Có được lồng ghép thỏa thuận thử việc trong hợp đồng lao động
 không?",
 "relevant_ids":["62671","61862"]}
```

Trong file này, `relevant_ids` chính là các document được xem là đúng đối với từng câu hỏi. Ngoài ra, `queries/example_queries.jsonl` chỉ chứa ba case rút gọn để đọc nhanh.

5.4 Cấu hình GraphRAG

Các thiết lập chính của GraphRAG được khai báo trong file `settings.yaml`. Mô hình ngôn ngữ được gọi qua OpenRouter, còn BGE-M3 được sử dụng thông qua một API embedding. Bạn có thể tự triển khai model BGE-M3 bằng FastAPI với mã nguồn `scripts/bge_m3_api.py` được cung cấp trong GitHub Repository ở cuối bài, sau đó cấu hình API này vào `settings.yaml`.

Dùng lệnh sau để khởi động API embedding:

```
python -m uvicorn scripts.bge_m3_api:app --host 0.0.0.0 --port 8000
```

Khi chạy trên cùng máy với GraphRAG, base URL của dịch vụ là `http://127.0.0.1:8000/v1`. API nạp model `BAAI/bge-m3` và cung cấp endpoint `POST/v1/embeddings` theo giao thức tương thích OpenAI.

Các cấu hình khi đó gồm:

```
completion_models:
  default_completion_model:
    type: litellm
    model_provider: openai
    model: deepseek/deepseek-v4-flash
    api_key: ${OPENROUTER_API_KEY}
    api_base: https://openrouter.ai/api/v1
    call_args:
      temperature: 0

embedding_models:
  default_embedding_model:
    type: litellm
    model_provider: openai
    model: BAAI/bge-m3
    auth_method: api_key
    api_key: ${EMBEDDING_API_KEY}
    api_base: http://127.0.0.1:8000/v1
    retry:
      type: exponential_backoff
      max_retries: 5
    call_args:
      timeout: 120

input:
  type: csv
  storage:
    type: file
    base_dir: corpus
```

```
  file_pattern: "legal_labor\\.csv$"
  id_column: id
  title_column: title
  text_column: text

chunking:
  type: tokens
  size: 600
  overlap: 80

extract_graph:
  entity_types:
    - government_body
    - legal_document
    - organization
    - person
    - process
    - legal_concept
  max_gleanings: 1
  prompt: prompts/extract_graph.txt

vector_store:
  type: lancedb
  db_uri: output/lancedb
  index_schema:
    text_unit_text:
      vector_size: 1024
    entity_description:
      vector_size: 1024
    community_full_content:
      vector_size: 1024

embed_text:
  embedding_model_id: default_embedding_model
  batch_size: 16
  batch_max_tokens: 8192
  names:
    - text_unit_text
    - entity_description
    - community_full_content

local_search:
  completion_model_id: default_completion_model
  embedding_model_id: default_embedding_model
  top_k_entities: 10
  top_k_relationships: 10

basic_search:
  completion_model_id: default_completion_model
```

```
  embedding_model_id: default_embedding_model
  k: 5
```

API FastAPI ở trên không yêu cầu xác thực, nhưng LiteLLM vẫn nhận trường `api_key`. Vì vậy, file `.env` có thể đặt:

```
OPENROUTER_API_KEY=...
EMBEDDING_API_KEY=unused
```

Bài thực hành này sử dụng BGE-M3, có vector đầu ra 1024 chiều. Vì vậy, `vector_size` của các index trong `vector_store` được đặt bằng 1024. Nếu sử dụng một mô hình embedding khác, cần thay giá trị này cho phù hợp với số chiều vector của mô hình đó.

Văn bản được chia thành các chunk có kích thước 600 token và overlap 80 token. Danh sách `entity_types` định hướng LLM tập trung vào các loại thực thể phù hợp với dữ liệu pháp luật.

Trong standard indexing pipeline, GraphRAG tạo embedding cho ba loại nội dung: `text_unit_text`, `entity_description` và `community_full_content`. Các vector này được lưu trong LanceDB và được sử dụng trong quá trình truy xuất.

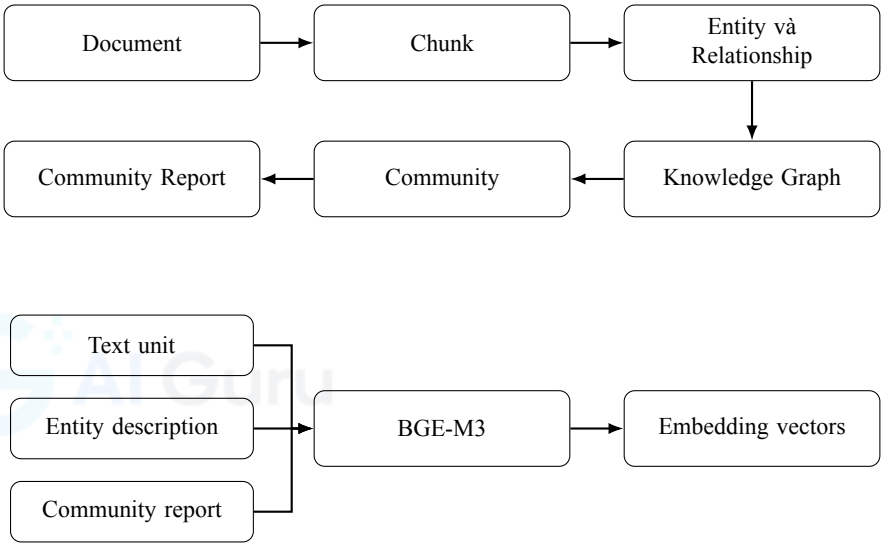
Đối với truy vấn, Local Search được cấu hình lấy tối đa 10 thực thể và 10 mối quan hệ, còn Basic Search lấy 5 đoạn văn bản có độ tương đồng cao nhất.

5.5 Lập chỉ mục cho dữ liệu

Sau khi chuẩn bị corpus và hoàn tất cấu hình, chúng ta bắt đầu quá trình indexing. Mục tiêu của bước này là chuyển các văn bản pháp luật ban đầu thành một đồ thị tri thức và dữ liệu phục vụ cho việc truy xuất.

Văn bản trước hết được chia thành các đoạn nhỏ, sau đó từng đoạn được phân tích để tìm các thực thể và mối quan hệ. Từ những kết quả này, GraphRAG mới xây dựng graph, phát hiện các community và tạo báo cáo cho từng community.

Về tổng quan, quá trình này có thể hình dung như Hình 2.

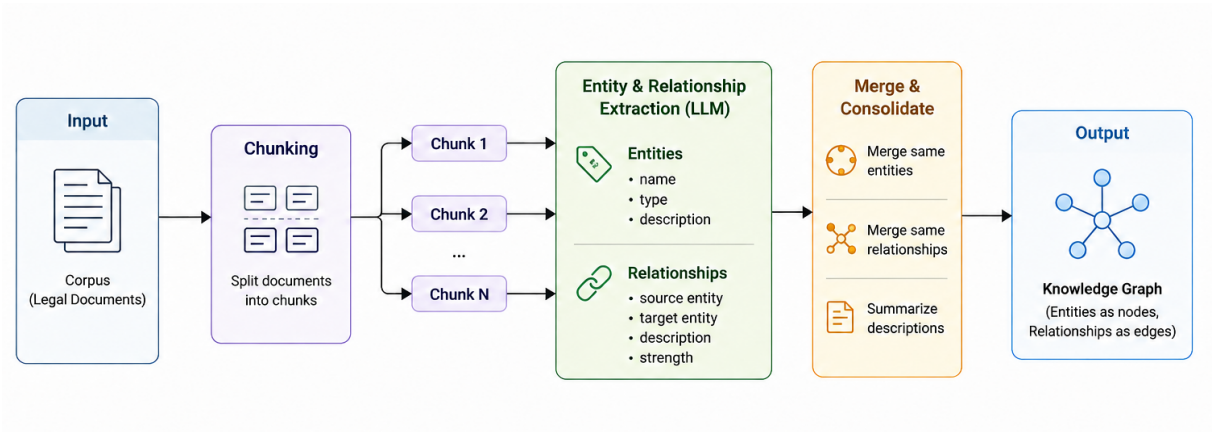


Hình 2: Luồng xây dựng graph và tạo embedding trong quá trình lập chỉ mục

5.5.1 Xác định thực thể và mối quan hệ

Trước hết, mỗi văn bản được chia thành các chunk. Các chunk này là đơn vị nội dung được đưa vào LLM để trích xuất thực thể và mối quan hệ.

Microsoft GraphRAG cung cấp sẵn prompt cho bước trích xuất các thành phần cho đồ thị. Trong bài này, prompt này được điều chỉnh để đầu ra sử dụng tiếng Việt, đồng thời giữ nguyên tên chính thức của các văn bản pháp luật, cơ quan và tổ chức.



Hình 3: Pipeline xây dựng đồ thị tri thức.

Bên cạnh đó, LLM cần được định hướng để tập trung vào những loại thực thể phù hợp với dữ liệu pháp luật. Khai báo các loại thực thể sau trong file `settings.yaml`:

Danh sách `entity_types` không xác định trước những thực thể cụ thể sẽ xuất hiện trong đồ thị. Nó chỉ giới hạn các nhóm mà LLM cần tập trung tìm kiếm, chẳng hạn cơ quan

nhà nước, văn bản pháp luật, tổ chức, cá nhân, quy trình và khái niệm pháp lý.

Với mỗi thực thể được phát hiện, LLM trả về ba thông tin chính:

- Tên thực thể;
- Loại thực thể;
- Mô tả về thực thể dựa trên nội dung của đoạn văn bản.

Ví dụ, một đoạn văn về Điều 24 của Bộ luật Lao động có thể đề cập đến "người lao động", "người sử dụng lao động", "thử việc" và "hợp đồng lao động". Từ nội dung này, LLM có thể nhận diện các đối tượng và khái niệm phù hợp để tạo thành các thực thể trong đồ thị.

Sau khi xác định các thực thể, LLM tiếp tục xem xét mối liên hệ giữa chúng. Nếu hai thực thể có quan hệ rõ ràng trong nội dung, mô hình tạo một *relationship* gồm thực thể nguồn, thực thể đích, phần mô tả mối quan hệ và mức độ liên hệ giữa hai thực thể.

Chẳng hạn, nếu một điều luật dẫn chiếu đến một điều luật khác, LLM có thể nhận diện hai điều luật thành hai thực thể và tạo một mối quan hệ mô tả việc dẫn chiếu giữa chúng.

Sau khi các chunk được xử lý, GraphRAG gộp các kết quả trích xuất để hình thành đồ thị chung. Nếu cùng một thực thể xuất hiện ở nhiều chunk với cùng tên và loại, các lần xuất hiện đó được gộp thành một thực thể duy nhất. Tương tự, các mối quan hệ có cùng thực thể nguồn và thực thể đích cũng được gộp lại.

Do một thực thể hoặc mối quan hệ có thể được mô tả khác nhau ở nhiều chunk, GraphRAG tiếp tục sử dụng LLM để tổng hợp các mô tả này thành một mô tả chung. Kết quả cuối cùng là một danh sách thực thể và mối quan hệ nhất quán hơn trên toàn corpus, trong đó các thực thể trở thành các node và các mối quan hệ trở thành các cạnh của đồ thị.

Cảnh báo:

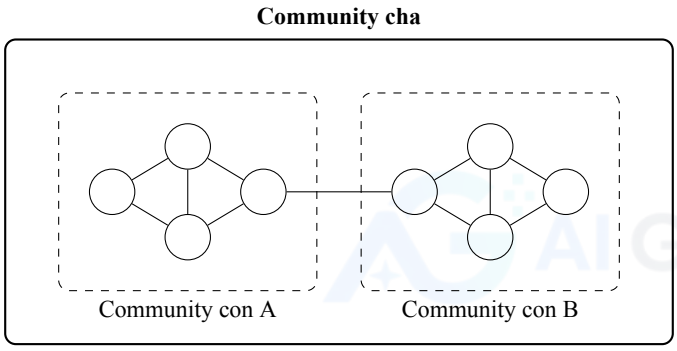
Thực thể và mối quan hệ trong graph là kết quả do LLM tạo ra, không phải dữ liệu đã được xác minh. Mô hình có thể sinh thực thể quá chung chung hoặc không có ý nghĩa, tách cùng một đối tượng thành nhiều thực thể, gộp nhầm các đối tượng khác nhau hoặc tạo quan hệ chỉ vì hai thực thể cùng xuất hiện trong một đoạn văn bản. Một relationship có mô tả hợp lý về mặt ngôn ngữ cũng chưa chắc được nội dung nguồn hỗ trợ. Hãy kiểm chứng và chuẩn hóa đồ thị tri thức trước khi đưa vào sử dụng.

5.5.2 Tạo Community từ đồ thị tri thức

Sau khi đã có đồ thị tri thức, Microsoft GraphRAG tiếp tục phân chia đồ thị thành các nhóm thực thể có liên kết chặt chẽ với nhau. Mỗi nhóm như vậy được gọi là một *community*.

Microsoft GraphRAG sử dụng thuật toán *Hierarchical Leiden* để phát hiện community trực tiếp từ cấu trúc của graph. Leiden xem xét cách các node kết nối với nhau và tìm cách phân chúng thành các nhóm sao cho các node trong cùng một nhóm có liên kết chặt chẽ hơn so với các node ở những nhóm khác.

Quá trình này tạo ra một hệ thống community theo nhiều cấp. Một community lớn có thể chứa nhiều community nhỏ hơn, nhờ đó cùng một graph có thể được quan sát ở các mức độ chi tiết khác nhau.



Hình 4: Minh họa cấu trúc community phân cấp trong knowledge graph

Trong Hình 4, các node trong mỗi nhóm có nhiều liên kết với nhau, trong khi số liên kết giữa hai nhóm ít hơn. Leiden sử dụng cấu trúc của toàn bộ graph để xác định cách phân nhóm phù hợp, thay vì dựa vào nhãn chủ đề có sẵn trong tài liệu.

Vì vậy, community không nhất thiết tương ứng với một document, một chương hay một chủ đề đã được định nghĩa trước. Các thực thể nằm ở nhiều văn bản khác nhau vẫn có thể thuộc cùng một community nếu chúng được kết nối với nhau trong graph.

Ngoài Leiden, bạn có thể tham khảo một số thuật toán phát hiện community phổ biến khác như *Louvain*, *Infomap*, *Label Propagation* và *Girvan–Newman*.

Lưu ý:

Bạn có thể tìm hiểu chi tiết hơn về cách Microsoft GraphRAG tạo community tại GraphRAG Indexing Dataflow.
Thuật toán Leiden được trình bày chi tiết trong bài báo *From Louvain to Leiden: guaranteeing well-connected communities*.

5.5.3 Community report được tạo như thế nào?

Sau khi các community được xác định, pipeline gọi LLM ở giai đoạn tạo báo cáo.

Với mỗi community, GraphRAG tập hợp các thực thể, mối quan hệ và nội dung liên quan rồi yêu cầu LLM viết một bản tổng hợp. Kết quả này được gọi là *community report*.

Có thể hiểu sự khác nhau như sau:

- **Community** cho biết một nhóm thông tin nào đang liên kết với nhau trong graph;
- **Community report** diễn giải và tổng hợp nội dung của nhóm thông tin đó thành văn bản.

Các community report đặc biệt quan trọng đối với Global Search. Thay vì phải đọc lại từng chunk riêng lẻ, Global Search có thể sử dụng các báo cáo này để tổng hợp thông tin từ nhiều khu vực của graph.

5.5.4 Thực hiện chạy bước lập chỉ mục

Ta không cần tự xây dựng lại pipeline lập chỉ mục từ đầu. Microsoft GraphRAG đã cung cấp sẵn *standard indexing pipeline*, bao gồm các workflow như chia văn bản thành chunk, trích xuất thực thể và mối quan hệ, xây dựng đồ thị, phát hiện community, tạo community report và tạo embedding.

Ta tận dụng trực tiếp pipeline này thông qua `PipelineFactory`:

```
from graphrag.config.load_config import load_config
from graphrag.index.run.run_pipeline import run_pipeline
from graphrag.index.workflows.factory import PipelineFactory


async def build(root):
    config = load_config(root_dir=root)

    # Lấy standard indexing pipeline của Microsoft GraphRAG
    pipeline = PipelineFactory.create_pipeline(
        config,
        "standard",
    )

    async for result in run_pipeline(pipeline, config):
        if result.error:
            raise result.error
```

Dòng

```
PipelineFactory.create_pipeline(config, "standard")
```

là phần quan trọng nhất. Nó tạo pipeline chuẩn đã được Microsoft GraphRAG định nghĩa sẵn. Vì vậy, các bước trích xuất entity và relationship, tạo community, tạo community report và `generate_text_embeddings` đều do GraphRAG thực hiện. Runner không xóa hay thay thế workflow embedding; điểm tùy chỉnh nằm trong `settings.yaml`, nơi embedding model được trỏ tới API BGE-M3.

Chạy quá trình lập chỉ mục bằng lệnh:

```
python scripts/run_index.py .
```

Nếu quá trình bị gián đoạn, có thể chạy lại cùng lệnh. GraphRAG sử dụng cache để tái sử dụng những kết quả đã hoàn thành, nhờ đó hạn chế việc gọi lại LLM cho những phần không thay đổi.

Nếu các bảng Parquet đã tồn tại và chỉ cần chạy lại workflow embedding, bạn có thể dùng:

```
python scripts/run_index.py . \
   --workflow generate_text_embeddings
```

5.5.5 Kết quả lập chỉ mục

Sau khi quá trình lập chỉ mục hoàn tất, 63 văn bản đầu vào được chia thành 68 text unit. Từ các đơn vị nội dung này, GraphRAG trích xuất 508 thực thể và 1.043 mối quan hệ để hình thành đồ thị tri thức. Đồ thị sau đó được phân chia thành 37 community, tương ứng với 37 community report.

Kết quả của lần chạy sử dụng trong bài thực hành được tóm tắt trong Bảng 3.

Thành phần	Số lượng
Văn bản	63
Text unit	68
Thực thể	508
Mối quan hệ	1.043
Community	37
Community report	37

Bảng 3: Kết quả lập chỉ mục của bài thực hành

Các con số trên cho phép kiểm tra nhanh toàn bộ chuỗi xử lý đã diễn ra như mong đợi về văn bản được phân đoạn, thực thể và mối quan hệ được trích xuất, đồ thị được hình thành và các community cùng community report đã được tạo.

Các kết quả này được lưu trong thư mục `output/`. Những file chính cần quan tâm gồm:

```
output/
 |-- documents.parquet
 |-- text_units.parquet
 |-- entities.parquet
 |-- relationships.parquet
 |-- communities.parquet
 |-- community_reports.parquet
 `-- stats.json
```

Các bảng Parquet tương ứng với những thành phần đã được tạo trong quá trình lập chỉ mục. `text_units.parquet` lưu các đoạn văn bản sau khi phân đoạn; `entities.parquet` và `relationships.parquet` lưu các nút và cạnh của đồ thị; `communities.parquet` và `community_reports.parquet` lưu kết quả phân nhóm và nội dung tổng hợp của từng community.

Lưu ý:

LLM được sử dụng để trích xuất thực thể, mối quan hệ và tạo community report. Do đó, số lượng thực thể, mối quan hệ và community có thể thay đổi giữa các lần chạy. Các giá trị trong Bảng 3 là kết quả của lần chạy được sử dụng trong bài thực hành, không phải các giá trị mà mọi lần chạy đều phải tái tạo chính xác.

5.6 Truy vấn với Local Search và Global Search

Sau khi indexing hoàn tất, GraphRAG có thể sử dụng dữ liệu đã tạo để xây dựng ngữ cảnh trả lời câu hỏi. Trong bước này, chúng ta thử hai phương thức truy vấn của Microsoft GraphRAG là Local Search và Global Search.

5.6.1 Local Search

Local Search bắt đầu bằng việc tìm những entity có nội dung liên quan đến câu hỏi. Từ các entity này, GraphRAG tiếp tục khai thác vùng lân cận trong graph, bao gồm các relationship, entity liên quan, chunk nguồn và những thông tin khác gắn với chúng.

Các dữ liệu thu được được chọn lọc để tạo thành context, sau đó context cùng câu hỏi được đưa vào LLM để sinh câu trả lời.

Ví dụ, với câu hỏi:

> "Có được ghi thỏa thuận thử việc trong hợp đồng lao động không?"

GraphRAG có thể tìm các entity liên quan đến "thử việc" và "hợp đồng lao động", sau đó lấy thêm các relationship và chunk nguồn xung quanh các entity này để bổ sung căn

cứ cho câu trả lời.

Chạy Local Search bằng lệnh:

```
graphrag query \
    "Có được ghi thỏa thuận thử việc trong hợp đồng lao động không?" \
    --root . \
    --method local
```

Local Search phù hợp với các câu hỏi tập trung vào một đối tượng, khái niệm, quy định hoặc mối quan hệ cụ thể trong graph.

5.6.2 Global Search

Global Search không bắt đầu từ một entity cụ thể. Phương thức này sử dụng các community report ở cấp community được chọn để tổng hợp thông tin trên phạm vi rộng hơn của corpus. Khác với Local Search, standard Global Search không cần bắt đầu bằng phép tìm entity gần nhất trong vector store.

Mỗi community report đã tóm tắt một vùng của knowledge graph. Khi nhận câu hỏi, GraphRAG xử lý nhiều report theo cơ chế *map–reduce*. Trong đó, ở bước *map*, từng nhóm community report cùng với câu hỏi được đưa vào LLM để rút ra các ý có liên quan và đánh giá mức độ quan trọng của chúng. Các ý quan trọng từ nhiều lượt map sau đó được gom lại thành một context mới. Ở bước *reduce*, context này được đưa vào LLM một lần nữa để tổng hợp thành câu trả lời cuối cùng.

Ví dụ, với câu hỏi:

> "Các nhóm nghĩa vụ chính của doanh nghiệp trong quan hệ lao động là gì?"

thông tin cần thiết có thể nằm ở nhiều community khác nhau, chẳng hạn các nhóm liên quan đến hợp đồng lao động, tiền lương, bảo hiểm xã hội hoặc quyền và nghĩa vụ của người sử dụng lao động. Global Search có thể tổng hợp các report của những community này thay vì chỉ mở rộng từ một entity ban đầu.

Chạy Global Search bằng lệnh:

```
graphrag query \
    "Các nhóm nghĩa vụ chính của doanh nghiệp trong quan hệ lao động là gì?" \
    --root . \
    --method global \
    --community-level 0 \
    --response-type "List of 3-5 points with source names"
```

Tham số `--community-level` xác định cấp community được sử dụng để lấy các report, còn `--response-type` quy định dạng câu trả lời mong muốn.

Global Search phù hợp với các câu hỏi cần tổng hợp thông tin từ nhiều phần khác nhau của corpus, thay vì tập trung vào một vùng nhỏ quanh một entity cụ thể.

6. Ví dụ khi GraphRAG có lợi thế

6.1 Case 1: Chữ ký điện tử và hợp đồng gửi qua mạng

Xét câu hỏi:

> Ảnh chữ ký của giám đốc trên bảng lương và hợp đồng gửi qua mạng có giá trị như chữ ký tay không?

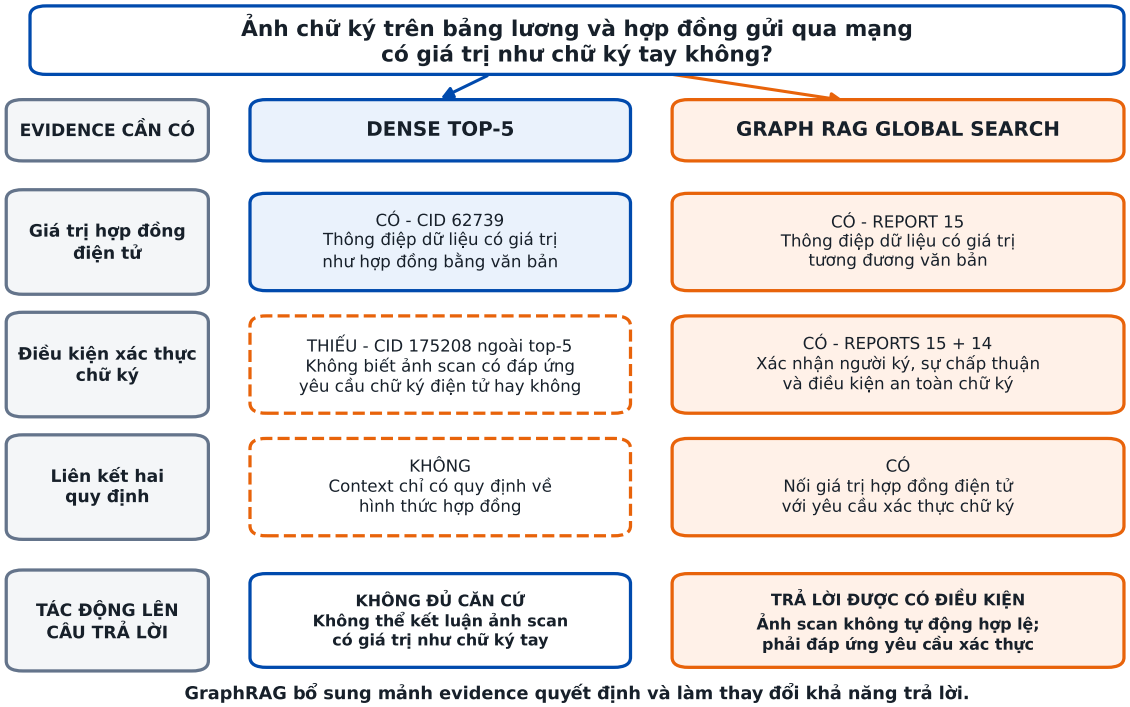
Để trả lời câu hỏi này, hệ thống cần tìm được hai nhóm thông tin khác nhau:

- quy định về giá trị pháp lý của hợp đồng được gửi dưới dạng thông điệp dữ liệu;
- điều kiện để chữ ký điện tử được xem là hợp lệ.

Với dense retrieval, các kết quả đầu tiên chứa quy định về hợp đồng được gửi qua phương tiện điện tử, nhưng chưa lấy được đầy đủ các điều kiện liên quan đến việc xác thực chữ ký. Vì vậy, ngữ cảnh cung cấp cho mô hình vẫn thiếu một phần căn cứ cần thiết để trả lời câu hỏi.

Global Search có thể tổng hợp thông tin từ nhiều cộng đồng trong đồ thị và đưa cả hai nhóm quy định trên vào cùng một ngữ cảnh. Nhờ đó, câu trả lời không chỉ xác định rằng hợp đồng điện tử có thể có giá trị pháp lý, mà còn chỉ ra rằng giá trị của chữ ký còn phụ thuộc vào các điều kiện xác thực tương ứng.

Trường hợp này minh họa lợi ích của GraphRAG khi câu hỏi cần kết hợp nhiều mảnh thông tin có liên quan nhưng nằm ở các phần khác nhau của corpus.



Hình 5: GraphRAG kết hợp các bằng chứng về hợp đồng điện tử và điều kiện xác thực chữ ký.

6.2 Case 2: Thử việc, BHXH và công đoàn

Xét câu hỏi:

> Người thử việc theo thỏa thuận riêng, chưa ký hợp đồng lao động chính thức, có phải đóng BHXH và có quyền vào công đoàn không?

Câu hỏi này liên quan đến ba nhóm thông tin khác nhau:

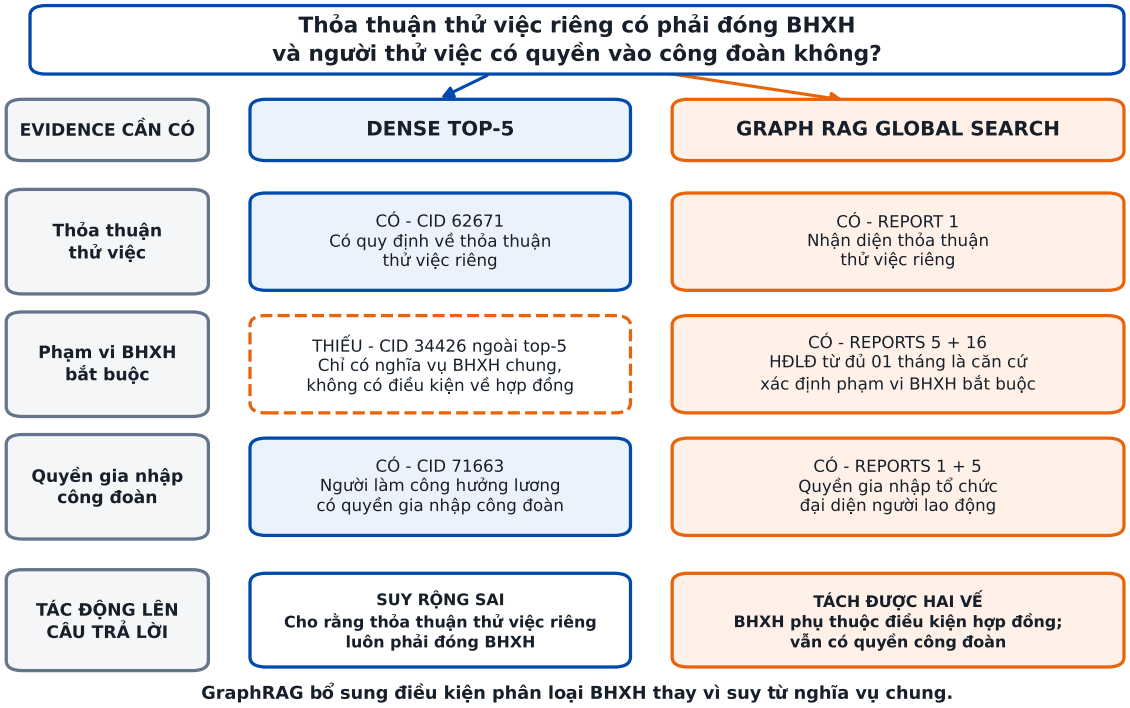
- quy định về thỏa thuận thử việc;
- điều kiện thuộc đối tượng tham gia BHXH bắt buộc;
- quyền gia nhập công đoàn của người lao động.

Với dense retrieval, hệ thống tìm được các văn bản liên quan đến thử việc và quyền gia nhập công đoàn, nhưng chưa lấy được đầy đủ quy định xác định trường hợp nào thuộc đối tượng tham gia BHXH bắt buộc. Vì vậy, nếu chỉ dựa trên các kết quả này, mô hình có thể suy rộng từ nghĩa vụ BHXH nói chung và đưa ra kết luận chưa chính xác cho trường hợp thử việc.

Global Search có thể tổng hợp thông tin từ các cộng đồng khác nhau trong đồ thị, qua đó đặt quy định về thử việc, điều kiện tham gia BHXH và quyền công đoàn trong cùng một

ngữ cảnh. Nhờ đó, hệ thống có thể phân biệt rằng nghĩa vụ BHXH còn phụ thuộc vào loại và thời hạn của quan hệ hợp đồng, trong khi quyền gia nhập công đoàn được xem xét theo một căn cứ khác.

Trường hợp này cho thấy GraphRAG có lợi thế khi một câu hỏi gồm nhiều vế và mỗi vế cần được đối chiếu với một nhóm quy định khác nhau.



Hình 6: GraphRAG kết hợp các nhóm thông tin về thử việc, BHXH và quyền gia nhập công đoàn.

6.3 Case 3: Hợp đồng mang tên "cộng tác viên"

Xét câu hỏi:

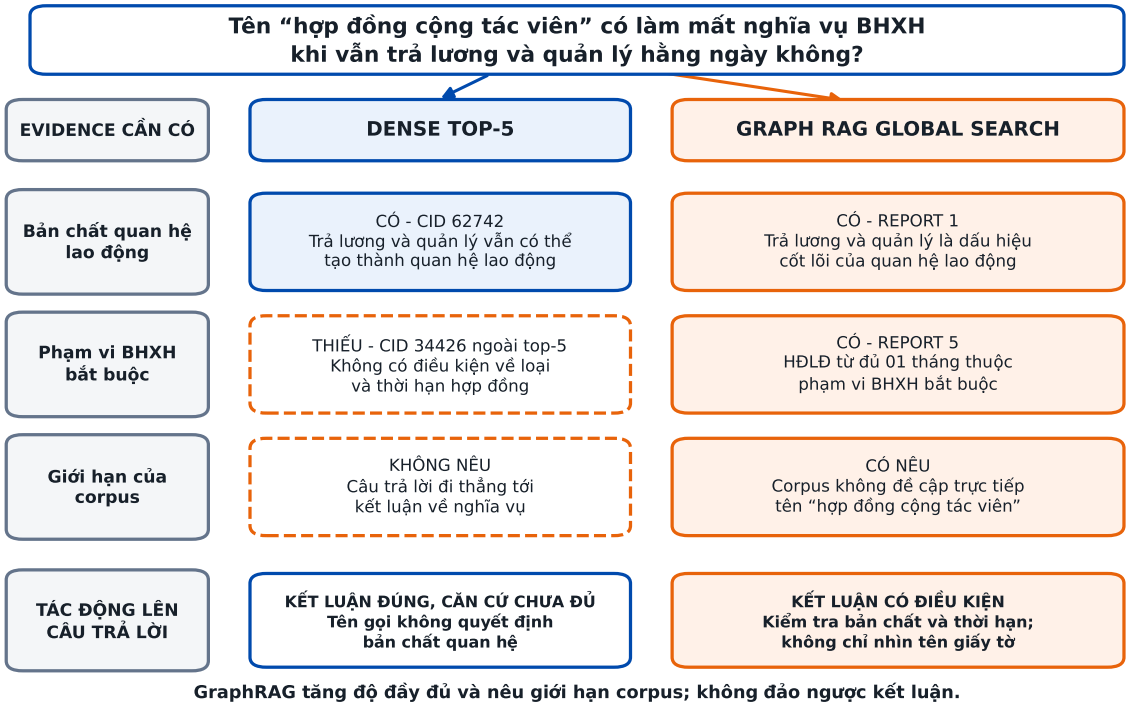
> Gọi giấy tờ là hợp đồng cộng tác viên nhưng vẫn trả lương và quản lý hằng ngày có làm mất nghĩa vụ đóng BHXH không?

Để trả lời câu hỏi này, hệ thống cần thực hiện hai bước. Trước hết phải xác định bản chất của quan hệ giữa hai bên dựa trên các dấu hiệu như trả lương và sự quản lý trong quá trình làm việc. Sau đó mới đối chiếu quan hệ đó với các điều kiện tham gia BHXH bắt buộc.

Dense retrieval tìm được quy định giúp xác định bản chất của quan hệ lao động, nhưng chưa lấy được đầy đủ căn cứ về phạm vi tham gia BHXH. Vì vậy, câu trả lời có thể đi đúng hướng nhưng vẫn thiếu một phần căn cứ để nối từ việc xác định quan hệ lao động đến nghĩa vụ BHXH.

Global Search có thể kết hợp hai nhóm thông tin này trong cùng một ngữ cảnh: một nhóm dùng để xác định bản chất quan hệ lao động và một nhóm quy định về đối tượng tham gia BHXH. Đồng thời, kết quả cũng cho thấy corpus không đề cập trực tiếp đến tên gọi "hợp đồng cộng tác viên", nên không thể chỉ dựa vào tên của giấy tờ để đưa ra kết luận.

Trường hợp này minh họa khả năng của GraphRAG trong việc nối các bước suy luận liên tiếp khi căn cứ cần thiết nằm ở những phần khác nhau của corpus.



Hình 7: GraphRAG kết hợp thông tin về bản chất quan hệ lao động và điều kiện tham gia BHXH.

7. Đánh giá khả năng truy xuất

Phần này đánh giá ảnh hưởng của thông tin từ đồ thị đối với khả năng truy xuất văn bản. Hai phương pháp được so sánh gồm:

- **Dense**: truy xuất trực tiếp trên toàn bộ tập văn bản bằng độ tương đồng giữa vector biểu diễn của câu hỏi và văn bản;
- **Graph + RRF**: bổ sung một nhánh truy xuất dựa trên community của GraphRAG, sau đó hợp nhất kết quả của hai nhánh bằng *Reciprocal Rank Fusion* (RRF).

Tập đánh giá gồm 26 câu hỏi tình huống. Các câu hỏi được xây dựng sau khi khảo sát

tập văn bản, vì vậy kết quả chỉ được sử dụng để phân tích hai phương pháp truy xuất trong phạm vi bài thực hành, không được xem là kết quả trên một benchmark độc lập.

7.1 Phương pháp truy xuất

Dense

Với mỗi câu hỏi, BGE-M3 tạo vector biểu diễn cho câu hỏi và so sánh với vector của 63 văn bản trong tập dữ liệu. Các vector được chuẩn hóa L2, do đó có thể sử dụng tích vô hướng để tính cosine similarity.

Các văn bản được sắp xếp theo độ tương đồng giảm dần và lấy 10 kết quả có điểm cao nhất.

Graph + RRF

Phương pháp Graph + RRF sử dụng các community được tạo trong quá trình lập chỉ mục để bổ sung thêm các văn bản liên quan. Với mỗi câu hỏi, quá trình truy xuất gồm các bước:

1. So sánh vector của câu hỏi với vector của các community report và chọn report có độ tương đồng cao nhất;
2. Xác định community tương ứng với report được chọn, sau đó lấy các văn bản đóng góp nội dung cho community này;
3. Xếp hạng các văn bản trong community bằng độ tương đồng với câu hỏi và giữ tối đa 10 kết quả;
4. Hợp nhất danh sách này với 10 kết quả của Dense bằng *weighted Reciprocal Rank Fusion*.

Entity và relationship không được truy xuất trực tiếp trong phương pháp này. Vai trò của chúng được thể hiện gián tiếp thông qua các community được hình thành từ cấu trúc entity–relationship của đồ thị.

RRF hợp nhất các danh sách dựa trên thứ hạng thay vì so sánh trực tiếp điểm của hai phương pháp. Với một văn bản đứng ở vị trí r trong một danh sách, phần điểm mà danh sách đó đóng góp được tính theo:

$$\frac{w}{k+r},$$

trong đó w là trọng số của nhánh truy xuất và k là hằng số làm giảm mức chênh lệch giữa các vị trí xếp hạng. Nếu một văn bản xuất hiện ở cả hai nhánh, hệ thống nhận diện theo

cùng `cid`, chỉ giữ một bản và cộng điểm từ hai thứ hạng.

Ở đây, k được đặt bằng 60, là giá trị thường được sử dụng với RRF. Trọng số của Dense được đặt bằng 1,0, còn nhánh community được đặt bằng 0,5. Việc giảm trọng số của nhánh community nhằm giữ Dense là nguồn xếp hạng chính, trong khi thông tin từ graph đóng vai trò bổ sung các văn bản có thể bị bỏ sót.

Tham số	Giá trị
Số kết quả từ Dense	10
Số community report được chọn	1
Số văn bản lấy từ community	Tối đa 10
Hằng số RRF k	60
Trọng số Dense	1,0
Trọng số nhánh community	0,5
Độ sâu đánh giá	10

Bảng 4: Cấu hình của phép đánh giá truy xuất

7.2 Thực hiện đánh giá

Mã đánh giá được đặt tại `scripts/evaluate_retrieval.py`. Bạn có thể chạy bằng lệnh:

```
python scripts/evaluate_retrieval.py . \
    --queries scripts/generated_candidate_queries.jsonl \
    --prefix generated
```

Kết quả được lưu trong thư mục `evaluation/`:

- `generated_metrics.json`: các chỉ số trung bình và độ trễ;
- `generated_per_query.csv`: kết quả chi tiết của từng câu hỏi;
- `generated_graph_wins.json`: các câu hỏi mà Graph + RRF đạt Recall@5 cao hơn Dense.

7.3 Chỉ số đánh giá

Ba chỉ số được sử dụng:

- **Recall@k**: tỷ lệ văn bản liên quan được tìm thấy trong k kết quả đầu tiên;
- **MRR@10**: phản ánh vị trí của văn bản liên quan đầu tiên trong 10 kết quả đầu. Văn bản liên quan xuất hiện càng sớm thì giá trị MRR càng cao;
- **nDCG@10**: đánh giá chất lượng thứ hạng của các văn bản liên quan trong 10 kết quả đầu.

7.4 Kết quả

Phương pháp	Recall@1	Recall@5	Recall@10	MRR@10	nDCG@10
Dense	**0,3205**	0,7532	**0,9135**	**0,8333**	**0,7804**
Graph + RRF	0,2949	**0,8205**	**0,9135**	0,7821	0,7650

Bảng 5: Kết quả truy xuất trung bình trên 26 câu hỏi tình huống

Graph + RRF đạt Recall@5 là 0,8205, cao hơn mức 0,7532 của Dense. Kết quả cho thấy việc bổ sung văn bản từ community giúp tìm được nhiều văn bản liên quan hơn trong năm kết quả đầu tiên.

Tuy nhiên, Dense đạt Recall@1, MRR@10 và nDCG@10 cao hơn. Trong phép thử này, Graph + RRF cải thiện độ bao phủ ở top-5 nhưng chưa cải thiện chất lượng xếp hạng tổng thể.

8. Tổng kết

Trong bài viết này, chúng ta đã xây dựng một pipeline Microsoft GraphRAG hoàn chỉnh trên 63 văn bản pháp luật lao động tiếng Việt. Từ corpus ban đầu, hệ thống đã xử lý văn bản, tạo đồ thị tri thức và các thành phần liên quan để sử dụng các dữ liệu này trong quá trình truy xuất.

Vì entity, relationship và community report chịu ảnh hưởng trực tiếp từ output của LLM, đừng vội tin tưởng graph ngay sau khi lập chỉ mục xong. Hãy dành thời gian mở vài thực thể, vài quan hệ ra xem và đối chiếu với văn bản gốc trước khi để hệ thống trả lời. Nếu áp dụng lên một tập văn bản lớn hơn nhiều so với dữ liệu trong bài này, bạn nên thử nghiệm lại trên chính dữ liệu của bạn, vì lợi ích của GraphRAG thường chỉ rõ ràng khi corpus đủ lớn và đủ phức tạp.

Toàn bộ mã nguồn của bài viết này được cung cấp tại GitHub Repository:

`https://github.com/Hoanghung0603/RAG_Advanced_GraphRAG.git`

Nguồn tham khảo

1. Microsoft GraphRAG, Getting Started: `https://microsoft.github.io/graphrag/get_started/`
2. Microsoft GraphRAG, CLI Reference: `https://microsoft.github.io/graphrag/cli/`

3. Microsoft GraphRAG, Detailed Configuration: https://microsoft.github.io/graphrag/config/yaml/
4. Microsoft GraphRAG, Indexing Methods: https://microsoft.github.io/graphrag/index/methods/
5. Microsoft GraphRAG, Input Data: https://microsoft.github.io/graphrag/index/inputs/
6. Microsoft GraphRAG, Query Overview: https://microsoft.github.io/graphrag/query/overview/
7. Microsoft GraphRAG, Global Search: https://microsoft.github.io/graphrag/query/global_search/
8. Microsoft GraphRAG source repository: https://github.com/microsoft/graphrag
9. YuITC, Vietnamese-Legal-Documents dataset: https://huggingface.co/datasets/YuITC/Vietnamese-Legal-Documents